

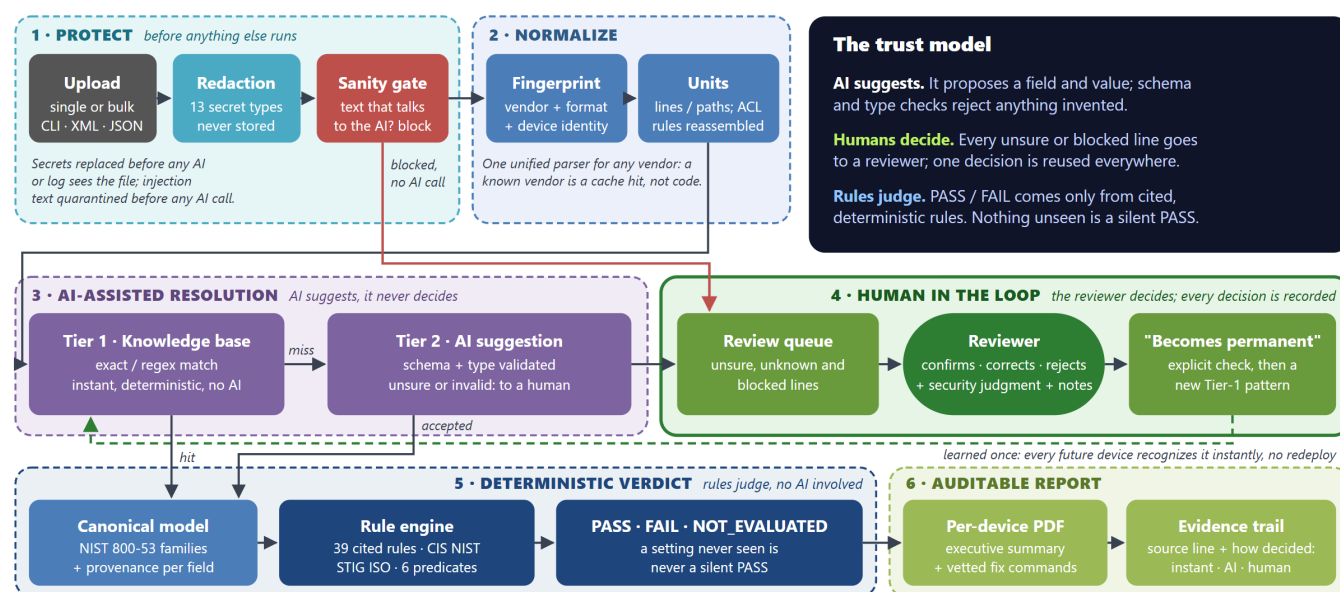
AEGIS: Architecture Document

Automated Evaluation & Governance for Infrastructure Security: an AI-augmented, vendor-agnostic network compliance engine

The problem, and the one design decision that answers it

Network devices from dozens of vendors must be checked against CIS, NIST SP 800-53, DISA STIG and ISO/IEC 27001, but every vendor's syntax differs, and a hardcoded parser per vendor breaks on the next vendor, firmware or device class. **AEGIS has one resolution path for every vendor: a lookup against one knowledge base (KB).** Only format handling is code, and it is generic (CLI lines, brace- and block-structured CLI, XML, JSON). A vendor is *data*: one YAML file with its fingerprint and patterns. Juniper Junos and FortiGate were added exactly that way, with no code. Unknown syntax is taught by a reviewer and becomes a KB row, never new code or a redeploy.

Pipeline



- 1. Redaction first.** Secret-shaped values (enable/user hashes, type-7 and plain passwords, SNMP communities and SNMPv3 keys, IPsec PSKs, RADIUS/TACACS+ keys, routing keys, FortiOS `ENC` and Junos quoted secrets, XML/JSON secret fields) become typed placeholders before anything else runs. Originals live only in memory for one request: never stored, logged or sent to an AI. The type digit stays visible because rules need it.
- 2. Fingerprint + identity.** Vendor, format, hostname, model, serial, firmware (e.g. Cisco `license udi ... sn ...`), and the SHA-256 of the uploaded file. Unrecognized vendors are not rejected; they route through Tiers 2/3.
- 3. Units with context.** One unit per setting; nested CLI keeps its parent path (`set system services telnet`). Banner text and free-text fields are never classified.
- 4. Input-sanity gate.** Config text is attacker-influenceable. Any unit shaped like an instruction to an AI is quarantined to human review *before any AI call* and shown as a blocked attack. Deterministic by design (an LLM filter has the same injection surface). A second, structural layer: free-text fields (descriptions, remarks) never reach the AI at all.
- 5. Confidence-tiered resolution:**
 - **Tier 1:** exact/regex KB match: instant, deterministic, including known non-security structure.
 - **Tier 2:** an AI model proposes {field, value, confidence} for ~12 lines per call, in parallel, cached (8-17 s per fresh file on free tiers). Groq then Gemini, or **your own model** (AEGIS_LLM_MODE=local, air-gapped), or none (off). Output is schema- and type-validated: invented fields or a value whose type contradicts the field are rejected.
 - **Tier 3:** anything rejected, unsure or unavailable goes to a reviewer. A confirmation becomes a new pattern, recorded as *human-confirmed*, and is hash-chained (`/audit/verify` detects any later edit).
- 6. Canonical model** on NIST 800-53 families (AC/AU/IA/SC/CM), so CIS, STIG and ISO share one schema. When a setting appears several times, the least secure value counts, and every source line is kept as evidence.

7. **Deterministic rule engine:** no AI in the verdict path. Six predicate types; ACLs are evaluated in order, per list, and only lists the device applies. A deny must cover all the traffic in question; an unreadable line before the decision yields *unknown*, never PASS. **39 rules** across the four frameworks. **Asymmetric trust:** AI-derived evidence may FAIL a control but never PASS a CAT I control without a human; a vendor's documented default (when the config is silent) may FAIL a rule, never PASS one.
8. **Remediation + PDF.** Fixes come only from vetted templates, filled with the device's own failing ACL and wrapped as a paste-in change. The PDF shows an integrity panel (input SHA-256, rule-set fingerprint, findings digest), the evidence line behind every finding, how it was decided, and why anything is unknown.

What makes it defensible, not just functional

- **Fail-closed by construction.** `NOT_EVALUATED` is a first-class verdict: a setting that is absent, unreadable, awaiting review or only AI-read is never a pass.
- **Every trust boundary is enforced in code.** Secrets redacted before inference; injection text never reaches the model; model output type-checked; verdicts deterministic; reviewer decisions validated, attributable (optional reviewer tokens) and tamper-evident.
- **Measured, errors included.** 73 verdicts hand-labelled from the config text over 7 configs and 5 vendors: **73 of 73 correct, 0 false PASS or FAIL**, with the AI off or on; CI fails the build on any wrong verdict. Prompt injection: the gate catches 32/32 tuned and **12/20 held-out** variants with 0 false positives on 416 real lines, and **0 of 52** attacks reach the AI end to end. Classifier: 95.8% precision on auto-accepted mappings (33-line golden set). An independent adversarial review found false-PASS paths in an earlier build; every one is now a regression test (242 offline tests in CI).

Tech stack

FastAPI (Python 3.11) · SQLAlchemy on SQLite (Postgres + pgvector is the production path) · local `sentence-transformers` embeddings · Groq `qwen3.8-27b` + Gemini 2.5 Flash, or any OpenAI-compatible local model · ReportLab · Langfuse · Next.js 16 + Tailwind · pytest + GitHub Actions.

Honest scope of this build

Not built yet: cross-reference linking by name (Cisco AAA method lists; Cisco ACL bindings, SONiC ACL rows, pfSense rules and FortiGate policies already are); vendor defaults beyond Cisco IOS; per-interface CDP (proxy ARP is per interface); entropy-based redaction; a login UI and roles; a job queue; live collection (Netmiko). Rule coverage is a representative 39-rule slice. ISO/NIST results are device-level evidence supporting a control, not a certification; CAT levels on CIS/NIST/ISO rules are AEGIS-assigned on the STIG scale. Local-model mode is covered by tests, not yet benchmarked on a real local model.